大规模skill场景下的问题与产品方案

一.现状与背景

1.现有的Agent skill管理系统的对比

首先，我收集了Open AI、Claude的skill管理方法，如下表所示：

核心模块 / 对比维度	OpenAI 的做法	Claude 的做法
Skill 注册 / 创建	• 支持直接创建、可视化编辑器创建以及文件/包上传。 • 注重平台化引导与开箱即用的工作流配置体验。	• 本质是文件夹体系，以 SKILL.md 为核心规范，可挂载 scripts/、references/、assets/ 等资源。 • 提供官方辅助 Skill（skill-creator）；同时支持通过 REST API（/v1/skills）及 Claude Console 控制台进行开发与注册。
Skill 组织与分类	• 采用标签化/视图化层级归类： - 已安装（Installed） - 我创建（Created by me） - 与我共享（Shared with me） - 工作空间共享（Workspace shared）	• 采用“文件系统 + 分层架构”进行深度组织。 • 基于元数据（name、description 等）统一声明；复杂业务逻辑支持下钻拆分为多个模块与外部参考脚本文件。
Skill 检索与发现	• 基于 Skill 管理界面的集中式页面检索与多维筛选（分类标签、名称过滤等）。	• 采用“渐进式披露”（Progressive Disclosure）机制： - 启动阶段预先加载轻量元数据（name + description）； - 任务意图匹配后再按需解析完整 SKILL.md 及深层资源脚本，极大节约上下文预算并提升命中精度。
Skill 调度与调用	• 由 ChatGPT 根据用户输入的任务指令与上下文语义，自主判断并决策是否激活调用对应 Skill。	• 上下文自适应触发，用户通常无需手动切换选择。 • 具备高灵活性，支持多个独立 Skill 在复杂复合任务中动态协同组合与流水线调用。
Skill 更新与版本	• 维护主体分层：创建者（Author）负责业务内容修改，组织管理员（Admin）统一把控审核与状态管理。	• 开发者级接口支持：提供开放的 /v1/skills API，支持以代码方式管理 Skill 的版本迭代与状态升级。 • 控制台支持：可在 Claude Console 中进行可视化查看、管理与升级发布。
权限管理	• 细粒度分级鉴权机制： - 用户级（User level） - 群组级（Group level） - 工作空间级（Workspace level）	• 依产品形态深度集成： - 本地与开发者侧遵循工程环境授权； - Claude Team / Enterprise 企业租户中，由组织管理员实施组织级（Organization-wide）全局启用与合规授权策略。
共享与分发	• 支持灵活的受众范围分发：可定向共享给指定人员、特定协同群组，或面向整个工作空间（Workspace）广播发布。	• 开发者端（Claude Code）：支持通过插件系统/插件市场（Marketplace）分发，或基于 Git 版本控制仓库在团队内协同共享。 • 企业端：由管理员在组织内统一部署及下发权限。

核心模块 / 对比维度	OpenAI 的做法	Claude 的做法
全生命周期管理	• 闭环流程明确： 创建 → 安装 → 使用 → 更新 → 删除	• 体系化渐进流程： 定义规范（SKILL.md） → 分发挂载（Marketplace/Git/配置） → 渐进式匹配与调度 → 版本平滑演进（API/Console） → 停用/归档

可以发现，现有 Agent Skill 体系已经完成从“无技能 Agent”到“可扩展技能 Agent”的转变，下一阶段瓶颈不再是 Skill 创建，而是大规模 Skill Library 下的检索效率、冲突管理和生命周期治理。

无论是Open AI的标签化/视图化层级归类，还是Claude的文件系统 + 分层架构，其分类目的都是为了人可以更快的检索到自己需要检索的skill，也仅仅是为了人方便查看。而私认为，技能的组织与分类应该实现检索的高效化和低成本化。

Claude 的 Progressive Disclosure 已经将 Skill 的完整 Instructions 延迟到触发后加载，从而降低了上下文成本。但其第一级 metadata（name + description）仍然需要在启动时加载。随着 Skill Library 从数十/数百扩展到数千甚至更大规模，所有 Skill metadata 都进入 system prompt 是否会形成新的上下文和检索效率瓶颈，是一个值得验证的问题。

二.问题的出现以及分析

长对话下的答非所问

【原因分析：时序滞后响应（Off-by-one Turn Lag)】

在长对话场景下，AI的准确率会大幅降低（包括chat GPT也会在对话达到一定长度后进行提示），我接下来会试图进行原因分析与对策的构想

以该段与Gemini的对话为例https://docs.google.com/document/d/1ZZ1gq1I1B7SQ4rXWsBu_vDkaY6EXApfIhnthgZDIuHk/edit?usp=sharing

尽管并没有长对话，但是在极短的对话中连续两次出现答非所问的问题足以说明该情况普遍，让我们看看Gemini对这种情况的解释：https://docs.google.com/document/d/1TFMLPWrB-jcx8gBf03EOHcZk-zl5hPqU29kA6IbOM1E/edit?usp=sharing

我们可以注意到，无论是第一点的技能会话状态未及时重置（Skill Session State Carryover）还是第三点的Skill 缺乏主动中断与重路由机制（Interrupt & Rerouting)，本质上都是因为系统缺乏对旧技能何时终止的有效判定与控制。最简单的办法是加入一个skill，在每次对话结束后强制终止Agent有关上次对话的所有旧技能。但是，毫无疑问这种方法会浪费算力：当用户的下一个指令与上一个指令高度相关时，重复激活skill实在没有必要。

因此，我们应该建立一个skill，用以判断新旧对话是否强相关，即是否有必要强制终止上次对话的所有旧技能。

方法如下：

Agent通过进行结构自闭环测试（Zero-Context Test）判断新指令是否具备独立可执行性：比如新任务为：“解释你说的内容”“不对吧”等时，Agent无法根据这句单独的话完成指令，即可认为上下两项指令强相关。

参数与槽位继承性（Slot & Parameter Dependency）即检查新指令是否在修改、复用或补充上一个技能所需的参数。

技能能力边界匹配度（Skill Scope / Capability Match)，即判断旧技能是否还有处理该指令的能力。

（3）我想到但被否定的方案：

直接对比前后指令的关键词以及需要调用的skill，有重复的就留下使用。

问题：为了知道新指令需要哪个 skill，系统必须把技能库里的上万个技能（N个）重新遍历、检索、打分一遍，算出新集合，再去和旧集合求交集，得不偿失。

Meta-skill：模仿Claude的Record a skill，即直接录制自己完成一个任务的过程，让 Claude 根据观察到的操作生成 Skill。让Agent收集管理员对前后文是否强相关的判断，进而生成对应的skill。

问题：成本过大。

skill冲突：技能库膨胀导致的选择低效

【原因分析:“重名互斥”、“语义踩踏”（Skill Collision & Shadowing）与检索选择低效】

当技能库足够大时，难免出现skill“重复”的现象（此处“重复”并不代表skill完全一样，那并无大碍，而是指大量功能极其相似的技能），甚至会因为语义微名片的相似度极高而发生技能踩踏，出现“权重高但并不适合的技能长期霸占调用权，更适配的垂直技能永远无法被召回”的问题。

对于此问题，我的想法是从skill的注册、调度、更新三方面进行改进，方法如下：

注册：注册时微名片进行向量相似度计算，防止微名片过于宽泛导致权重高但并不适合的技能长期霸占调用权。【防碰撞排他检查（Registration Collision Gate）：如果发现与现有技能相似度高于 0.90，系统会拒绝直接上线，并强制要求开发者填写消歧专有属性（Disambiguation Features）或负向触发条件（Negative Triggers）】

调度：每个月强制使用“冷门技能”，如将10%的技能使用留给未使用过的/新开发的skill。

更新：

统计skill在不同领域指令的覆盖率，如果发现一个skill在多个领域的指令中同时出现，可怀疑其出现语义踩踏，进行人工核验。

统计报错率，高于设定值则检查，可能是微名片不准确。（并不代表只有这一种可能，也有可能是skill本身出现了问题等）

skill更新不及时带来的不便

【原因分析：参数幻觉、接口模式漂移（Schema Drift & Parameter Hallucination）与外部接口变化不匹配】

当Agent需要使用外部软件的skill时，可能面临API更新而skill原本记录为原有api，导致无法顺利接入。对此，我从检查问题和解决问题两步提出想法：

允许Agent在真实 Docker 沙箱中进行预先探索，检查API是否更改，如果更改，及时进行更新或者下线该技能。

当执行任务时发现API更改，先尝试反向追踪API更改前的来源，再在其中尝试寻找更改后的API。（https://docs.google.com/document/d/1rsgFYlkESG_pzqukgze-II3b0F0HSnBeWcSF0Orskvo/edit?usp=sharing）

用户与需求

从技术问题到用户需求：我们考虑到对这些问题影响到的用户以及他们的需求进行分析，于是从以下五个方面入手。

1.用户画像

从痛点推导具体需求

典型使用场景

痛点一：Skill 太多，不容易找到真正合适的（问题2）

痛点二：Skill 信息可能过时，找到以后也不一定能正常使用（问题3）

痛点三：多轮对话以后，Skill 的状态和调用可能不稳定（问题1）

具体使用场景如下：

场景一：skill注册

在skill开发者注册skill时，往往在skill library已经存在多个功能相似的skill，此时就面临需要检查是否存在冲突并规范优化skill的情况。

场景二：Skill 检索

作为Skill使用者自身，我们知道我们的主要目的是让 Agent 在真实任务中可靠完成任务，也就是说我们在乎的核心是Skill 是否能正确使用。但常常在多轮对话和连续任务中当skill过多，agent检索skill library时，面临大量相似skill，可能选错skill，这是用户在skill检索时面临的痛点。

场景三：长对话

对于skill用户另一个重要的点在于skill使用的稳定性，历经长对话后，继续补充要求和其他信息，或者换其他任务，常常会出现信息错乱，理解偏差，错误继承和调用的问题。

场景四：API 更新

Skill创建之后，如果API发生变化，而skill描述没有及时更新，agent就会按照旧的指令去调用，此时就会出现错误。

这三部分表格总结如下：

目标用户	使用场景	核心痛点	用户需求
Skill 开发者	管理大量 Skill	不知道 Skill 是否过期、接口是否还能用	快速检查 Skill 健康状态
Skill 开发者	新增 Skill	Skill 信息不完整、边界不清晰	自动生成/检查 Metadata
Skill 开发者	Skill 数量增长	重复、冲突、相似 Skill 太多	Skill 冲突/重复检测
Agent 使用者	长对话任务	Skill 使用可能不稳定	检查长对话下 Skill 是否仍有效
Skill 管理者	Skill 库维护	难以持续维护大量 Skill	风险报告/维护建议

对调研发现和待验证假设的区分

当前可以确认的发现

OpenAI / Claude 已经有 Skill 管理机制

Progressive Disclosure 已经存在

大规模 Skill 管理存在组织和检索问题

Skill 更新涉及版本和接口维护

内容	类型	当前依据	下一步验证
Skill 数量增加后，重复/相似 Skill 管理难度增加	调研发现	Skill Library / 相关研究	用户访谈 / Skill 库分析
Skill 文档与实际接口可能出现不一致	待验证假设	技术场景分析	构造 Skill 测试
长对话可能导致 Skill 使用稳定性下降	待验证假设	已有研究线索	多轮对话实验
用户需要查看 Skill 的能力边界	产品假设	Negative Boundary 设计	用户测试
用户需要 Skill 健康检查	产品假设	痛点推导	原型测试

还有很多其他值得思考的问题：

- Skill 数量增加到什么规模以后 Metadata 会真正成为瓶颈?
- 0.90 相似度是否真的合理?
- 长对话多少轮以后错误率明显增加?
- Schema Drift 等问题在真实 Skill 中出现的频率是多少?

Moscow

功能模块	功能举例 / 说明	优先级	定位说明	规划原因 / 依据
Skill 基础信息检查（metadata）	名称、描述、参数、更新信息、能力边界	Must (必需)	没他产品基本不能用（核心基础）	MVP核心
Skill 版本/更新时间检查	—	Must (必需)	没他产品基本不能用（核心基础）	对应痛点1
Skill 链接/API可用性检查（skill health check）	—	Must (必需)	没他产品基本不能用（核心基础）	判断Skill是否“健康”
长对话测试	—	Must (必需)	没他产品基本不能用（核心基础）	对应痛点3
风险报告	—	Must (必需)	没他产品基本不能用（核心基础）	用户需要看到结果
Skill 冲突检测（Skill Conflict Detection）	重复 Skill、相似 Skill、功能冲突、名称冲突	Should (应该)	很重要但第一版可以没有	对应痛点2，但暂不作为核心
自动修改 metadata	—	Could (可以)	有了更好（次要体验）	过于复杂
自动更新 skill	—	Could (可以)	有了更好（次要体验）	过于复杂

功能模块	功能举例 / 说明	优先级	定位说明	规划原因 / 依据
自动修复 Skill	—	Could (可以)	有了更好（次要体验）	过于复杂
自动重新训练模型	—	Won't (不做)	先不做	超出MVP范围

产品方案

AI-assisted Skill Indexing + Human-in-the-loop Governance的构想

当skill library中的skill数量增加到极大规模时，单纯依靠Progressive Disclosure的检索方法也会在检索过程中消耗大量时间以及算力，为了弥补这一缺陷，我提出在Progressive Disclosure检索之前加入分类构建技能索引系统（Skill Index），具体构想如下：

注册：技能冲突检测（Skill Conflict Detection）

注册时微名片进行向量相似度计算，防止微名片过于宽泛导致权重高但并不适合的技能长期霸占调用权。【防碰撞排他检查（Registration Collision Gate）：如果发现与现有技能相似度高于 0.90，系统会拒绝直接上线，并强制要求开发者填写消歧专有属性（Disambiguation Features）或负向触发条件（Negative Triggers）】

组织：技能索引（Skill Index）

技能完成注册后，系统利用语义表征模型（Semantic Embedding Models）解析其元数据描述，并与技能库（Skill Repository）中的现有实体进行向量相似度计算。当相似度或语义关联度达到预设阈值时，将其归入对应的技能索引（Skill Index）。分类体系尽可能多维，可以从业务语义领域（如气象、地理空间等领域上下文）与工具依托类型（如文档处理、演示文稿生成等操作载体）等角度分类。为适配复杂任务场景，系统采用多标签分类（Multi-label Classification）与重叠聚类（Overlapping Clustering）机制，允许单一技能同时从属于多个语义范畴或功能类簇。

检索与调度：通过技能索引+渐进式披露进行检索

系统接收用户任务后，智能体解析指令语义意图并映射至功能类簇空间，再在对应类别中依托渐进式披露机制完成细粒度匹配与按需激活。

针对现实场景中普遍存在的跨领域复合型任务，优先考虑在这些索引中同时出现的skill。在保障任务语义契合度的同时压缩候选集规模，显著降低上下文窗口的 Token 预算开销。

值得注意的是：当技能库足够大时，难免出现“重名互斥”和“语义踩踏”的现象，而复合型任务通常需要垂直领域的专家型skill组合协作，而非一个跨界的“万金油” Skill。跨多类别的 Skill 往往定义泛化、准确率低，更容易引发幻觉，对于此问题我会在接下来的“维护与改进”中展开。

更新：人工参与的管理机制 （Human-in-the-loop Governance）

由AI自主完成分类毫无疑问会出现问题，因此还需要设立人工检测，具体工作流程如下：

设立报错机制

我将报错检测分为三个等级，对应优化方案也有所不同

Skill级：即skill本身出现问题

维护：每月对skill报错率进行统计，报错率高于设定值则进行更改、优化

Cluster级：即skill分类出现问题

原因分析

命名体系不统一：从注册层面解决

技能的能力范围（Scope）定义过宽，产生“语义贪婪”：从注册层面解决

Retrieval级：即对skill选择出现问题（skill的检索效率低）

原因分析：

skill重复出现，降低检索效率，智能体不知道该选择哪一个

构想：使用反面示例（Negative Examples）：当系统内的技能越来越庞杂时，相似技能会导致模型选错。通过在技能定义中加入明确的负面规则（例如“别在 X 场景调用我，这时请用 Y 技能”），能大幅恢复并提升技能的正确触发率。

智能体在无特意制作的精选技能时难以自己通过通用技能完成任务

根本原因：context-prompt过程的缺点：当用户提供的文本中没有关键词时，Agent如何完成任务？

构想：联想能力，即对context进行联想获得可能出现的关键词，将这些关键词自由组合，再在数据库中比对得到关键词可能一起出现的概率最高的情况作为智能体的任务（即各分别任务的连带关系）

后续维护：对用户反馈或报错的词汇收集并定期处理，出现频率高于某个值的单独构建精选skill

检索系统分类混乱

构想：设计最高报错率，当某个技能出现在某个类别的报错率高于最高报错率时，人工干预检查该技能是否真的属于这个类别。

建立对应skill完成报错及报错检测

模仿Claude的Record a skill，即直接录制自己完成一个任务的过程，让 Claude 根据观察到的操作生成 Skill。让Agent收集管理员人工修改、完善skill的数据，生成对应的Meta-skill。

（样本过大？难度过大？有待解决）

模仿Open AI的skill creator功能，设定skill对报错率进行监测，一旦报错率高于设定值，激活上文中生成的Meta-skill来对skill进行增、删、改。

五.MVP与验证